

Módulo 2

Como a Web Funciona e HTTP na Prática

FastAPI · Ambiente Python · Venv · Swagger · CORS

O que veremos hoje

1 Como a web funciona

Cliente, servidor e ciclo de requisição

2 Ambiente Python

venv, pip e requirements.txt

3 Primeira API FastAPI

Rotas, parâmetros e Swagger

4 HTTP na prática

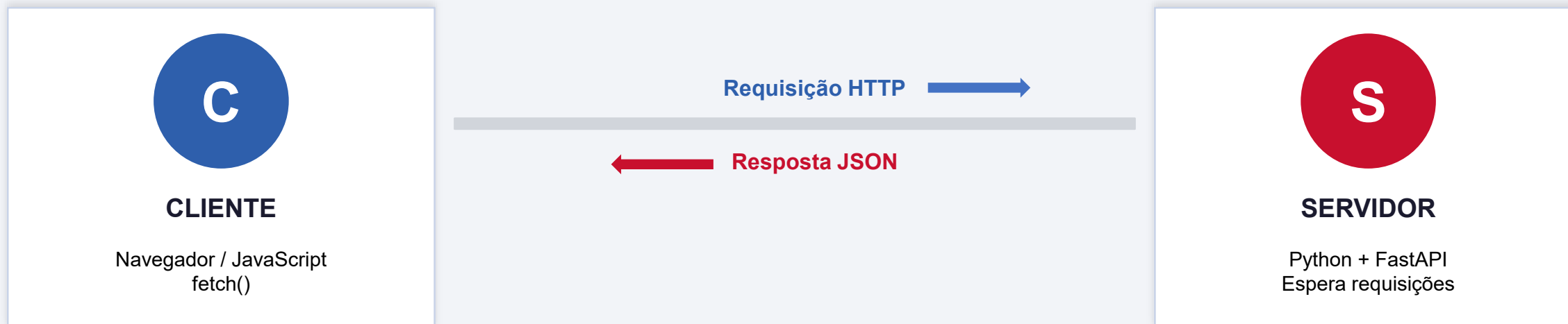
DevTools, status codes e testes

5 Integração front-back

fetch() + CORS na prática

O que é um servidor web?

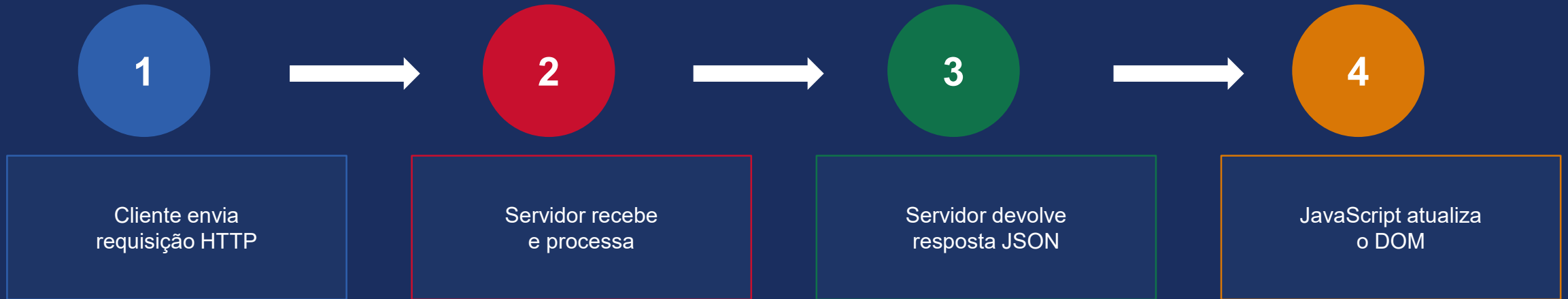
Bloco 1 — Como a web funciona



Módulo 1: vocês eram o CLIENTE — o navegador que fazia requisições para APIs externas.

Módulo 2: vocês são também o SERVIDOR — o programa Python que recebe e responde.

Ciclo de uma Requisição HTTP



Requisição contém:

- **Método:** GET, POST, PUT, DELETE...
- **URL:** `http://localhost:8000/usuarios`
- **Headers:** Content-Type, Authorization...
- **Body (opcional):** dados em JSON

Resposta contém:

- **Status code:** 200, 201, 404, 500...
- **Headers:** Content-Type, CORS...
- **Body:** dados em JSON
- **Ex:** `{"id": 1, "nome": "Carlos"}`

Por que FastAPI?

Comparação com outros frameworks Python

Flask

✓ **Simples e popular**

X Validação, docs e async precisam de libs extras

Django

✓ **Completo e robusto**

X Curva de aprendizado alta para APIs simples

NOSSA ESCOLHA

FastAPI

✓ **Moderno, rápido, tipado**

✓ **Tipagem Python nativa**

✓ **Swagger automático**

✓ **Validação com Pydantic**

✓ **Alta performance**

Ambiente Virtual (venv)

Bloco 2 — Cada projeto tem o seu próprio ambiente isolado

SEM venv

Python Global

Projeto A: fastapi==0.95

Projeto B: fastapi==0.104

Projeto C: fastapi==?

⚠ Conflito de versões!

COM venv ✓

Projeto A (venv próprio)

✓ fastapi==0.95

Projeto B (venv próprio)

✓ fastapi==0.104

✓ Sem conflitos!

Comandos essenciais:

```
python -m venv venv          # cria o ambiente virtual
venv\Scripts\activate        # ativa no Windows (venv) aparece no terminal
deactivate                   # desativa quando terminar
```

Instalando Dependências

1 Instalar FastAPI e Uvicorn

```
pip install fastapi uvicorn  
  
# Verificar instalação  
pip list
```

2 Salvar as dependências

```
pip freeze > requirements.txt  
  
# Instalar em outra máquina  
pip install -r requirements.txt
```



FastAPI

O framework — define rotas, valida dados, gera documentação automaticamente.



Uvicorn

O servidor ASGI — fica "escutando" as requisições e executa a API.

Analogia: FastAPI é o motor do carro. Uvicorn é a estrada e o combustível que fazem o motor rodar.

Estrutura do Projeto e .gitignore

Estrutura esperada em aula05/

```
aula/
  venv/           ← NO .gitignore!
  main.py         ← código da API
  requirements.txt ← dependências
  .gitignore      ← arquivos ignorados
  teste-front.html ← teste de integração
```

Conteúdo do .gitignore:

```
venv/
__pycache__/
*.pyc
.env
```

⚠ Nunca commite a pasta venv/

- X Pode ter centenas de megabytes
- X É facilmente recriável com o requirements.txt
- X Contém binários específicos do sistema operacional
- X Cada dev recria o próprio venv localmente

✓ Fluxo para quem clonar o projeto

```
git clone URL-do-repositorio
cd modulo2-fastapi/aula05
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```


A Primeira API com FastAPI

Bloco 3 — Criar, rodar e testar

```
# main.py -- Primeira API com FastAPI
from fastapi import FastAPI

app = FastAPI(
    title='API de Cadastro -- SENAI',
    version='0.1.0'
)

# Decorador @app.get('/') define a rota
@app.get('/')
def raiz():
    return {'mensagem': 'API funcionando!', 'versao': '0.1.0'}

@app.get('/status')
def status():
    return {'status': 'online', 'servico': 'API SENAI'}
```

Anatomia de uma rota

```
@app.get('/rota')
```

Decorador: define o método HTTP e a URL

```
def nome_funcao():
```

Função Python normal que processa a requisição

```
return {...}
```

Dicionário Python → JSON automático

```
uvicorn main:app --reload
```

→ <http://localhost:8000> · /docs para o Swagger · --reload reinicia ao salvar

Rotas com Parâmetros

Path Parameter

Faz parte da URL entre chaves { }

```
@app.get('/usuarios/{usuario_id}')
def buscar_usuario(usuario_id: int):
    for u in usuarios_db:
        if u["id"] == usuario_id:
            return u
    return {'erro': 'Não encontrado'}
```

Exemplos de URL

```
GET /usuarios/1      → retorna Carlos Silva
GET /usuarios/99     → retorna erro (não existe)
GET /usuarios/abc    → erro 422 automático!
```

Query Parameter

Aparece depois do ? na URL

```
@app.get('/usuarios/busca')
def buscar_por_nome(nome: str = ''):
    if not nome:
        return usuarios_db
    return [u for u in usuarios_db
            if nome.lower() in u['nome'].lower()]
```

Exemplos de URL

```
GET /usuarios/busca      → todos
GET /usuarios/busca?nome=ana → Ana Lima
GET /usuarios/busca?nome=dev → Desenvolvedor
```

Swagger UI — Documentação Automática



Gerado automaticamente

Zero configuração. Só criar as rotas com FastAPI.



Testar sem ferramentas

Try it out → Execute direto no browser.



Schemas visíveis

Modelos de entrada e saída documentados.



Curl command

Gera o comando curl de cada requisição.

Acesse em:

`http://localhost:8000/docs`

→ Swagger UI (documentação interativa)

`http://localhost:8000/redoc`

→ ReDoc (documentação alternativa)

Status Codes HTTP

Bloco 4 — Cada código comunica exatamente o que aconteceu

200 OK

GET bem-sucedido

201 Created

Criado com POST

204 No Content

Sucesso sem body (DELETE)

400 Bad Request

Dados inválidos enviados

401 Unauthorized

Token ausente ou inválido

403 Forbidden

Sem permissão de acesso

404 Not Found

Recurso não encontrado

422 Unprocessable Entity

Validação Pydantic falhou

500 Internal Server Error

Erro inesperado no servidor

CORS — Cross-Origin Resource Sharing

⚠ O Problema

Live Server: localhost:5500

Nossa API: localhost:8000

Portas diferentes = origens diferentes

→ CORS bloqueado pelo navegador!

✓ A Solução

Configurar o CORSMiddleware no FastAPI:

```
from fastapi.middleware.cors import
    CORSMiddleware
app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
)
```

```
# Adicionar no main.py após criar o app
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],    # em produção: especificar o domínio
    allow_methods=['*'],
    allow_headers=['*'],
)
```

Integração Front-Back

Bloco 5 — JavaScript consumindo a nossa própria API

```
// teste-front.html - fetch() para a API FastAPI
const response = await fetch('http://localhost:8000/usuarios');
const usuarios = await response.json();

const lista = document.querySelector('#lista');
lista.innerHTML = usuarios.map(u => `
  <div class="card">
    <span class="nome">${u.nome}</span>
    <span class="status ${u.ativo ? "ativo" : "inativo"}">
      ${u.ativo ? "Ativo" : "Inativo"}
    </span>
  </div>
`).join("");
```

Resultado no browser:

Carlos Silva

Ativo

Desenvolvedor

Ana Lima

Ativo

Designer

Bruno Costa

Inativo

QA

✓ Front-end (Módulo 1) + Back-end (Módulo 2) integrados! Os dados vêm da nossa própria API Python.

Atividade Prática

Expandindo a API de usuários

Adicionar em main.py:

GET

GET /usuarios/ativos

→ retorna só os com ativo == True

GET

GET /usuarios/inativos

→ retorna só os com ativo == False

GET

GET /usuarios/cargo/{cargo}

→ filtra por cargo (case-insensitive)

GET

GET /info

→ total, ativos e inativos (dict)

Requisitos

- ✓ List comprehension nos filtros
- ✓ Anotação de tipo: cargo: str
- ✓ /info retorna dicionário
- ✓ Testar no Swagger
- ✓ Commit após tudo funcionar

Commit: atividade novas rotas fastapi → Push origin → Link no Google Classroom

Resumo da Aula



Como a web funciona

Ciclo: cliente → requisição → servidor → resposta JSON



Ambiente Python

venv isola dependências · pip install · requirements.txt



FastAPI + Uvicorn

uvicorn main:app --reload · rotas com @app.get()



Swagger automático

localhost:8000/docs · Try it out · sem configuração



CORS configurado

CORSMiddleware · front e back integrados



Próxima aula

Pydantic · validação de dados · POST e PUT